

DecoyAuth: Integration into Wi-Fi

Mathy Vanhoef

Abstract

This document provides a detailed specification of how the DecoyAuth protocol can be integrated into IEEE 802.11, i.e., into the Wi-Fi protocol. The integration of DecoyAuth enables the AP to detect the usage of a decoy password, indicating a data breach, for instance, that one of the access points got compromised. This also paves the way to enable clients to authenticate using any password from a given set of accepted passwords. This work was supported by NGI Sargasso.

1 Introduction

This document builds on the DecoyAuth protocol whose high-level description and design is given in [5]. Briefly summarized, DecoyAuth provided mutual authentication in a zero-knowledge fashion, with the advantage that it supports multiple authentication tokens or passwords. This enables the support of decoy tokens or password. The idea is that when a decoy token or passwords is used, this indicates to the Access Point (AP) that a data breach has occurred and that appropriate action needs to be taken. In other words, the decoy tokens function as a reverse honeypot: a decoy key is indistinguishable from a real key, and when an adversary steals or uses a decoy key, the counter-party can detect this [5]. Second, DecoyAuth also paves the way towards accept multiple valid authentication tokens or passwords. That is, as long as the client uses one of the accepted passwords, authentication will succeed, and the network will also learn which password was used. This enables the network to, for instance, place each different user into a separate Virtual LAN (VLAN).

The internals of DecoyAuth builds upon concepts from O-PAKE and Dragonfly [3, 1]. Our current version of DecoyAuth, in theory, supports any number of decoy passwords, with both computation and data overhead increasing linearly with the number of decoys employed. In practice, this linear overhead implies that there is a limit to the number of supported (decoy) passwords, to keep data and computational overhead manageable.

A proof-of-concept implementation of DecoyAuth, integrated into IEEE 802.11, can be found at <https://github.com/DistriNet/decoyauth/blob/main/hostap/README.md>. Although this proof-of-concept implementation is not guaranteed to precisely match the latest specification provided in this document, the high-level details behind the integration remain the same as described in this document.

2 High-Level Overview

The DecoyAuth protocol builds upon the Simultaneous Authentication of Equals (SAE) handshake, which is defined in the IEEE 802.11 standard and which is also used in WPA3. As a result, when integrating DecoyAuth into Wi-Fi, we can build upon the existing

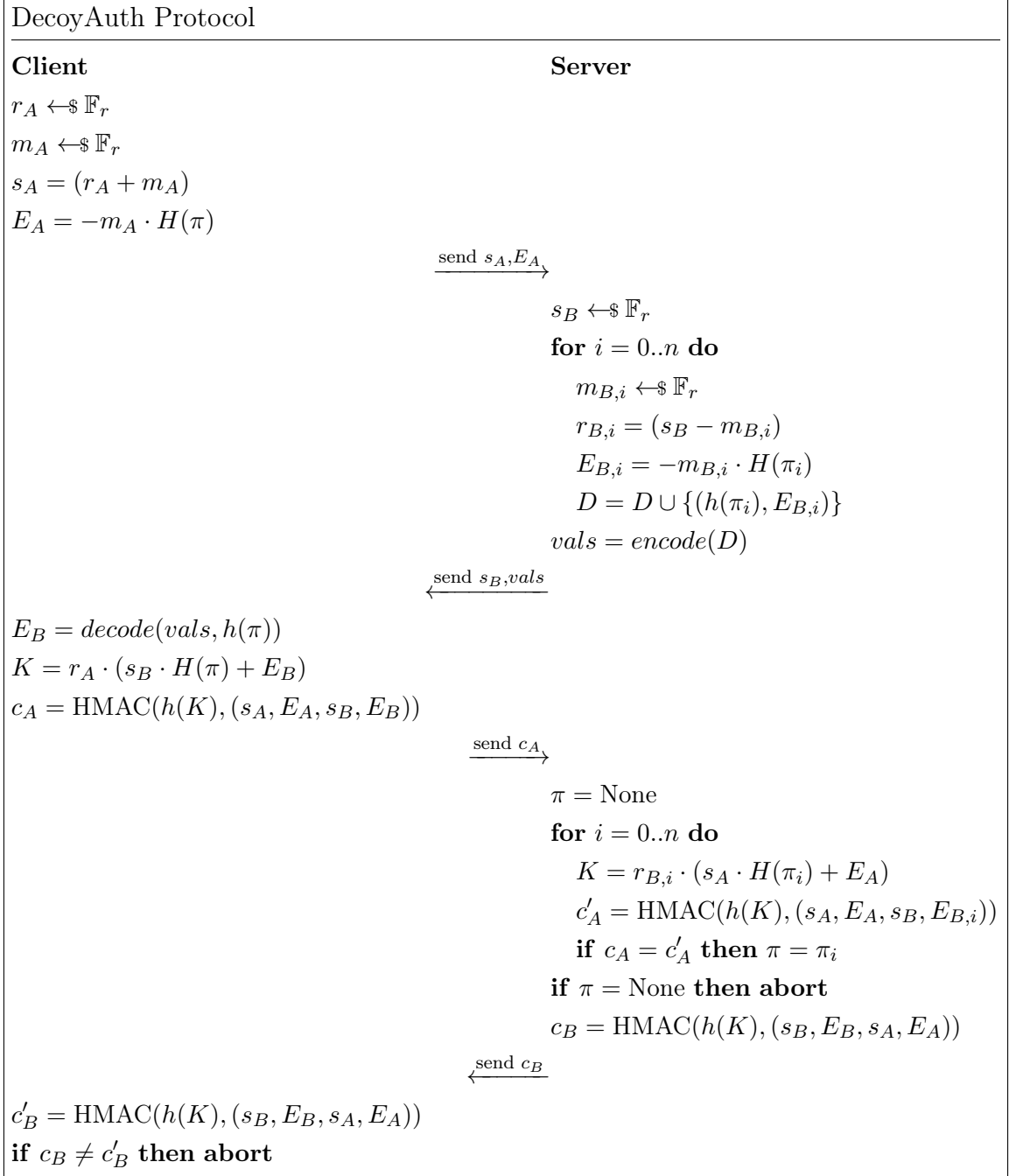


Figure 1: Design of the DecoyAuth protocol.

Commit and Confirm frame formats that are used in the Simultaneous Authentication of Equals handshake.

While SAE is defined as a symmetric protocol, where both peers can simultaneously initiate the handshake, and both peers perform exactly the same computations upon receiving a frame this is not the case DecoyAuth. This implies that, unlike SAE, we do not explicitly support mesh networks but instead focus on infrastructure networks where there is a central AP. In these infrastructure networks, the client always initiates a connection with this AP, here by sending an SAE Commit frame

When integrating the DecoyAuth protocol into IEEE 802.11, the first SAE Commit frame sent by the client remains identical as in the existing SAE protocol. The SAE Commit reply by the AP will contain *vals*, which is an encoded set of elliptic curve points, indexed by the hash of the corresponding password, one for each password, allowing the client to continue using only one particular password [5, §3.2]. Lastly, the format of SAE Confirm messages stays the same, but the AP will process the SAE Confirm from the client in a new manner.

Overall, this means the following changes to SAE have to be made to integrate the DecoyAuth protocol into IEEE 802.11: the format of the Commit frame sent by the AP needs to be updated, the generation of this frame thereby also needs to be updated, and the processing of two received frames needs updating. More precisely, this corresponds to the following updates:

- The format of the SAE Commit frame sent by the AP to the client needs to be updated such at it can contain *vals*, an encoded set of elliptic curve points for each allowed password (Section 3).
- The AP needs a new procedure to generate the updated SAE Commit frame in response to the client's SAE Commit frame (Section 4).
- The client needs updated procedures to process the modified SAE Commit frame from the AP, in particular, to use the hashed password to continue with the appropriate received parameters. Additionally, the AP, on receipt of the SAE Confirm frame, will have to loop over all accepted passwords to determine the password used by the client (Section 5).

Lastly, for ease of reference, Figure 1 repeats the description of the DecoyAuth protocol as specified in full detail in [5].

3 Frame Format Update

The existing SAE handshake in 802.11 uses management frames to transport handshake messages. Their default format, and the extensions to support DecoyAuth, are discussed in this section.

3.1 Default commit frame format

The body of such an authentication frame starts with a 1-byte authentication algorithm number, followed by a 1-byte authentication transaction sequence number (see Figure 2). For SAE messages, the former byte always equals 3 to represent that SAE is being used, and the value of the latter byte depends on whether the frame is a Commit or

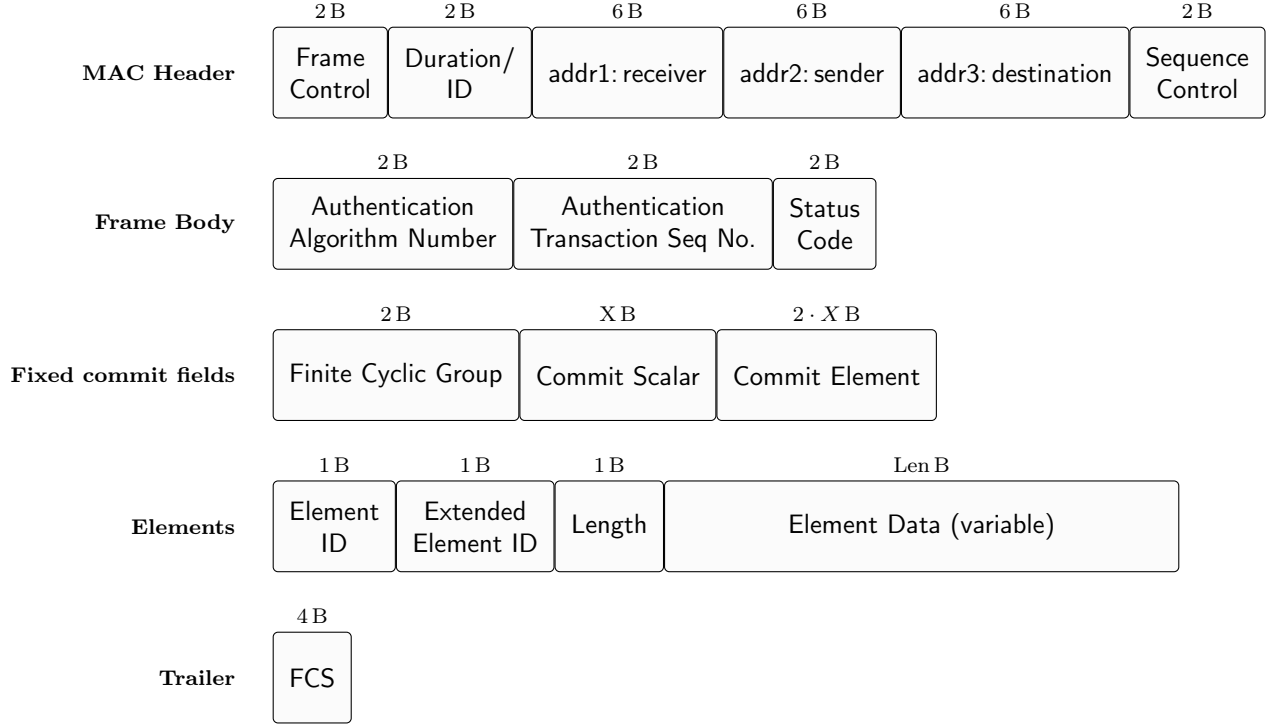


Figure 2: Format of the Commit frame sent by the AP. The integration of DecoyAuth adds a new element, whose corresponding Element ID and Extended Element ID is shown in Table 1.

Confirm frame. These fields are followed by a 2-byte status code field. The frame then contains several fixed-length fields whose presence depends on the status code, followed by a variable list of type-length-value elements, and finally a CRC checksum to detect transmission errors [2, Table 9-71].

For a traditional commit frame, the fixed-length fields in the middle are the 2-byte finite cycle group, the commit scalar, and the commit element. The finite cycle group represent which elliptic curve is used in the handshake, which in turn influences the length of the commit scalar and element. Briefly summarized, the commit scalar and commit element are fresh values in each handshake, where the commit scalar is a random number, and the commit element is a point on the chosen elliptic curve. Although SAE also supports groups other than elliptic curves, we assume that only elliptic curve are used, since they are also the most widely-supported cyclic group in IEEE 802.11 implementations.

3.2 Format extensions

To enable the transportation of the *vals* parameter, we can add a new type-length-value element near the end of the frame. Specifically, the type of this element would equal 255, indicating the usage of element ID extension, where the extended ID can have a currently unused value [2, Table 9-130]. The scalar value s_B used in DecoyAuth can be transported in the existing scalar field.

To keep backwards-compatibility with existing clients, we propose to populate the Commit Scalar and Commit Element fixed fields with a value corresponding to a “fallback

Table 1: Overview of the elements contained in the authentication Commit frame sent by the AP when integrated with support for the DecoyAuth protocol [2, §Table 9-130].

ID	Extended ID	Element Description
255	33	Password identifier
255	92	Rejected Groups
255	93	Anti-Clogging Token Container
255	114	AKM Suite Selector
255	136	Known STA Identification
255	144	Encoding of DecoyAuth’s <i>vals</i> parameters

password”, or to set this field to all-zeros. The AP could then either detect the usage of the fallback passwords during the handshake, or, when using an all-zero field, older clients will refuse the connect since this value is not within the allowed range.

We remark that in the current version of the standard, there is no support to fragment these frames, meaning their size is limited to roughly 2300 bytes (excluding MAC header). This means that, without further extending IEEE 802.11 with support to fragment management frames, this maximum frame size limits the amount of simultaneous passwords that can be supported in practice.

4 Frame Transmission Update

For the AP to send the updated frame described in the previous section, the *vals* parameters first need to be generated. When not using any additional optimizations, this needs to be done for every handshake initiated by the client. Specifically, the steps that the AP needs to perform are:

1. Generate a random scalar s_B , i.e., a random number in $[2, r[$ where r is the order of the elliptic curve.
2. For every supported (decoy) authentication token or password π_i , do:
 - (a) Generate a random scalar $s_{B,i} \in [2, r[$ and calculate $r_{B,i} = (s_B - m_{B,i})$. If the value $r_{B,i}$ is not in $[2, r[$ then generate a new random scalar value $s_{B,i}$.
 - (b) Calculate the field element $E_{B,i} = -m_{B,i} \cdot H(\pi_i)$. Note that this is a point on an elliptic curve.
 - (c) Add the pair (password hash, elliptic curve point) to the set D , that is, add $(h(\pi_i), E_{B,i})$ to the set D .
3. Now encode the pairs in D into the parameters *vals* as follows:
 - (a) Use the technique of Tibouchi [4] to securely encode the elliptic curve points, resulting in two values for each elliptic curve point. The results in two sets D_1 and D_2 .
 - (b) Use polynomial interpolation on both the sets D_1 and D_2 . There the password hash is treated as the x-coordinate and part of the encoded elliptic curve point as the y-value. The resulting coefficients of the polynomial equal the parameters $vals_1$ and $vals_2$.

After this calculation, the parameters *vals* can be transported in the new element defined in Table 1. Note that the receiver can infer the size of each polynomial coefficient from the chosen elliptic curve. Additionally, the receiver can infer the total number of coefficients from the total size of the total element field size.

5 Frame Receiving Updates

5.1 Client processing of commit frame

When the client receives the new commit frame of the AP, the client needs to use its password to obtain the appropriate parameters to continue the handshake. The following procedure is followed to accomplish this:

1. The client splits up the received *vals* field into two equal-length parts *vals*₁ and *vals*₂. Based on the selected elliptic curve group, and total size of the element field, the client can determine how many coefficients are present in both *vals*₁ and *vals*₂.
2. The client calculates $u = \text{decode}(\text{vals}_1, h(\pi))$ and $v = \text{decode}(\text{vals}_2, h(\pi))$. In other words, the hash of the client's password is treated as the x-coordinate, and the received *vals* are used to calculate the y-coordinate.
3. The client uses the methods of Tibouchi [4] to recover the original elliptic curve point E_B from u and v .

The SAE handshake that continues as normal with the received elliptic curve point E_B . There are no changes to the generation of the confirm frame towards the AP.

5.2 AP processing of confirm frame

On receipt of the confirm frame of the client, the AP needs to determine which password the client used. For this, the AP needs to iterate over all supported (decoy) passwords, recalculate the client's confirm value, and see whether it matches the received confirm value. If there is no match, this means the client used a wrong (decoy) password, and the connection is rejected.

If there is a match, this means the client used a supported (decoy) password. The AP can then generate the final response confirm value as usual based on the negotiated shared secret key.

6 Conclusion

The proposed DecoyAuth protocol supports decoy passwords while still ensuring strong security properties such as mutual authentication, forward secrecy, and resistance against off-line dictionary attacks. Regarding overhead, the server must loop through all passwords twice, and the data overhead is linear in function of the number of supported decoy passwords. Future work includes further optimizing the protocol and providing a formal analysis and proof.

Acknowledgements

This work was funded by NGI Sargasso, the Research Fund KU Leuven, and the Cybersecurity Research Programme Flanders.

References

- [1] Dan Harkins. Simultaneous authentication of equals: A secure, password-based key exchange for mesh networks. In *2008 Second International Conference on Sensor Technologies and Applications (sensorcomm 2008)*, pages 839–844. IEEE, 2008.
- [2] IEEE Std 802.11. *Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, 2024.
- [3] Franziskus Kiefer and Mark Manulis. Oblivious pake: Efficient handling of password trials. In *International Conference on Information Security*, pages 191–208. Springer, 2015.
- [4] Mehdi Tibouchi. Uniform points on elliptic curves of prime order as uniform random strings. In *International Conference on Financial Cryptography and Data Security*, 2014.
- [5] Mathy Vanhoef. DecoyAuth: Protocol design. Retrieved 11 September 2025 from <https://github.com/DistriNet/decoyauth/blob/main/docs/whitepaper.pdf>, 2025.